

Chat history

The chat history object is used to maintain a record of messages in a chat session. It is used to store messages from different authors, such as users, assistants, tools, or the system. As the primary mechanism for sending and receiving messages, the chat history object is essential for maintaining context and continuity in a conversation.

Creating a chat history object

A chat history object is a list under the hood, making it easy to create and add messages to.

C#

```
using Microsoft.SemanticKernel.ChatCompletion;

// Create a chat history object
ChatHistory chatHistory = [];

chatHistory.AddSystemMessage("You are a helpful assistant.");
chatHistory.AddUserMessage("What's available to order?");
chatHistory.AddAssistantMessage("We have pizza, pasta, and salad available to order. What would you like to order?");
chatHistory.AddUserMessage("I'd like to have the first option, please.");
```

Adding richer messages to a chat history

The easiest way to add messages to a chat history object is to use the methods above. However, you can also add messages manually by creating a new `ChatMessage` object. This allows you to provide additional information, like names and images content.

C#

```
using Microsoft.SemanticKernel.ChatCompletion;

// Add system message
chatHistory.Add(
    new() {
        Role = AuthorRole.System,
        Content = "You are a helpful assistant"
    }
);

// Add user message with an image
chatHistory.Add(
```

```
new() {
    Role = AuthorRole.User,
    AuthorName = "Laimonis Dumins",
    Items = [
        new TextContent { Text = "What available on this menu" },
        new ImageContent { Uri = new Uri("https://example.com/menu.jpg") }
    ]
}
);

// Add assistant message
chatHistory.Add(
    new() {
        Role = AuthorRole.Assistant,
        AuthorName = "Restaurant Assistant",
        Content = "We have pizza, pasta, and salad available to order. What would you like to order?"
    }
);

// Add additional message from a different user
chatHistory.Add(
    new() {
        Role = AuthorRole.User,
        AuthorName = "Ema Vargova",
        Content = "I'd like to have the first option, please."
    }
);
```

Simulating function calls

In addition to user, assistant, and system roles, you can also add messages from the tool role to simulate function calls. This is useful for teaching the AI how to use plugins and to provide additional context to the conversation.

For example, to inject information about the current user in the chat history without requiring the user to provide the information or having the LLM waste time asking for it, you can use the tool role to provide the information directly.

Below is an example of how we're able to provide user allergies to the assistant by simulating a function call to the `User` plugin.

Tip

Simulated function calls is particularly helpful for providing details about the current user(s). Today's LLMs have been trained to be particularly sensitive to user information. Even if you

provide user details in a system message, the LLM may still choose to ignore it. If you provide it via a user message, or tool message, the LLM is more likely to use it.

C#

```
// Add a simulated function call from the assistant
chatHistory.Add(
    new() {
        Role = AuthorRole.Assistant,
        Items = [
            new FunctionCallContent(
                functionName: "get_user_allergies",
                pluginName: "User",
                id: "0001",
                arguments: new () { {"username", "laimonisdumins"} }
            ),
            new FunctionCallContent(
                functionName: "get_user_allergies",
                pluginName: "User",
                id: "0002",
                arguments: new () { {"username", "emavargova"} }
            )
        ]
    }
);

// Add a simulated function results from the tool role
chatHistory.Add(
    new() {
        Role = AuthorRole.Tool,
        Items = [
            new FunctionResultContent(
                functionName: "get_user_allergies",
                pluginName: "User",
                id: "0001",
                result: "{ \"allergies\": [\"peanuts\", \"gluten\"] }"
            )
        ]
    }
);
chatHistory.Add(
    new() {
        Role = AuthorRole.Tool,
        Items = [
            new FunctionResultContent(
                functionName: "get_user_allergies",
                pluginName: "User",
                id: "0002",
                result: "{ \"allergies\": [\"dairy\", \"soy\"] }"
            )
        ]
    }
);
```

```
}  
);
```

Important

When simulating tool results, you must always provide the `id` of the function call that the result corresponds to. This is important for the AI to understand the context of the result. Some LLMs, like OpenAI, will throw an error if the `id` is missing or if the `id` does not correspond to a function call.

Inspecting a chat history object

Whenever you pass a chat history object to a chat completion service with auto function calling enabled, the chat history object will be manipulated so that it includes the function calls and results. This allows you to avoid having to manually add these messages to the chat history object and also allows you to inspect the chat history object to see the function calls and results.

You must still, however, add the final messages to the chat history object. Below is an example of how you can inspect the chat history object to see the function calls and results.

C#

```
using Microsoft.SemanticKernel.ChatCompletion;  
  
ChatHistory chatHistory = [  
    new() {  
        Role = AuthorRole.User,  
        Content = "Please order me a pizza"  
    }  
];  
  
// Get the current length of the chat history object  
int currentChatHistoryLength = chatHistory.Count;  
  
// Get the chat message content  
ChatMessageContent results = await chatCompletionService.GetChatMessageContentAsync(  
    chatHistory,  
    kernel: kernel  
);  
  
// Get the new messages added to the chat history object  
for (int i = currentChatHistoryLength; i < chatHistory.Count; i++)  
{  
    Console.WriteLine(chatHistory[i]);  
}
```

```
// Print the final message
Console.WriteLine(results);

// Add the final message to the chat history object
chatHistory.Add(results);
```

Chat History Reduction

Managing chat history is essential for maintaining context-aware conversations while ensuring efficient performance. As a conversation progresses, the history object can grow beyond the limits of a model's context window, affecting response quality and slowing down processing. A structured approach to reducing chat history ensures that the most relevant information remains available without unnecessary overhead.

Why Reduce Chat History?

- **Performance Optimization:** Large chat histories increase processing time. Reducing their size helps maintain fast and efficient interactions.
- **Context Window Management:** Language models have a fixed context window. When the history exceeds this limit, older messages are lost. Managing chat history ensures that the most important context remains accessible.
- **Memory Efficiency:** In resource-constrained environments such as mobile applications or embedded systems, unbounded chat history can lead to excessive memory usage and slow performance.
- **Privacy and Security:** Retaining unnecessary conversation history increases the risk of exposing sensitive information. A structured reduction process minimizes data retention while maintaining relevant context.

Strategies for Reducing Chat History

Several approaches can be used to keep chat history manageable while preserving essential information:

- **Truncation:** The oldest messages are removed when the history exceeds a predefined limit, ensuring only recent interactions are retained.
- **Summarization:** Older messages are condensed into a summary, preserving key details while reducing the number of stored messages.
- **Token-Based:** Token-based reduction ensures chat history stays within a model's token limit by measuring total token count and removing or summarizing older messages when the limit is exceeded.

A Chat History Reducer automates these strategies by evaluating the history's size and reducing it based on configurable parameters such as target count (the desired number of messages to retain) and threshold count (the point at which reduction is triggered). By integrating these reduction techniques, chat applications can remain responsive and performant without compromising conversational context.

In the .NET version of Semantic Kernel, the Chat History Reducer abstraction is defined by the `IChatHistoryReducer` interface:

C#

```
namespace Microsoft.SemanticKernel.ChatCompletion;

[Experimental("SKEXP0001")]
public interface IChatHistoryReducer
{
    Task<IEnumerable<ChatMessageContent>?>
    ReduceAsync(IReadOnlyList<ChatMessageContent> chatHistory, CancellationToken cancel-
    lationToken = default);
}
```

This interface allows custom implementations for chat history reduction.

Additionally, Semantic Kernel provides built-in reducers:

- `ChatHistoryTruncationReducer` - truncates chat history to a specified size and discards the removed messages. The reduction is triggered when the chat history length exceeds the limit.
- `ChatHistorySummarizationReducer` - truncates chat history, summarizes the removed messages and adds the summary back into the chat history as a single message.

Both reducers always preserve system messages to retain essential context for the model.

The following example demonstrates how to retain only the last two user messages while maintaining conversation flow:

C#

```
using Microsoft.SemanticKernel.ChatCompletion;
using Microsoft.SemanticKernel.Connectors.OpenAI;

var chatService = new OpenAIChatCompletionService(
    modelId: "<model-id>",
    apiKey: "<api-key>");

var reducer = new ChatHistoryTruncationReducer(targetCount: 2); // Keep system mes-
```

sage and last user message

```
var chatHistory = new ChatHistory("You are a librarian and expert on books about cities");

string[] userMessages = [
    "Recommend a list of books about Seattle",
    "Recommend a list of books about Dublin",
    "Recommend a list of books about Amsterdam",
    "Recommend a list of books about Paris",
    "Recommend a list of books about London"
];

int totalTokenCount = 0;

foreach (var userMessage in userMessages)
{
    chatHistory.AddUserMessage(userMessage);

    Console.WriteLine($">>> User:\n{userMessage}");

    var reducedMessages = await reducer.ReduceAsync(chatHistory);

    if (reducedMessages is not null)
    {
        chatHistory = new ChatHistory(reducedMessages);
    }

    var response = await chatService.GetChatMessageContentAsync(chatHistory);

    chatHistory.AddAssistantMessage(response.Content!);

    Console.WriteLine($">>> Assistant:\n{response.Content!}");

    if (response.InnerContent is OpenAI.Chat.ChatCompletion chatCompletion)
    {
        totalTokenCount += chatCompletion.Usage?.TotalTokenCount ?? 0;
    }
}

Console.WriteLine($"Total Token Count: {totalTokenCount}");
```

More examples can be found in the Semantic Kernel [repository](#) .

Next steps

Now that you know how to create and manage a chat history object, you can learn more about function calling in the [Function calling](#) topic.

Learn how function calling works

Last updated on 01/31/2025